Internet of Things 12 (2020) 100319

Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot

Lightweight and host-based denial of service (DoS) detection and defense mechanism for resource-constrained IoT devices

Jerry John Kponyo, Justice Owusu Agyemang*, Griffith Selorm Klogo, Joshua Ofori Boateng

Faculty of Electrical/Computer Engineering, Kwame Nkrumah University of Science and Technology, Kumasi, Ghana

ARTICLE INFO

Article history:
Received 16 March 2020
Revised 23 September 2020
Accepted 27 October 2020
Available online 6 November 2020

Keywords:
IoT
DoS
SYN flood
ICMP flood
UDP flood
Machine learning

ABSTRACT

The Internet of Things (IoT) is an intelligent network that connects smart objects to the Internet. A large number of IoT devices are connecting to the Internet, yet many of these devices are insecure, exposing them to a number of security threats. Most IoT devices are resource constrained hence making it difficult to secure them using conventional security techniques. A number of researchers have proposed intrusion detection mechanisms implemented at IoT gateways. IoT devices become vulnerable when the security mechanisms put in place at the gateway fails. This has opened a new research into how conventional security measures can be applied to IoT devices through the development of lightweight and host-based security measures. This paper presents a lightweight and host-based detection and defense mechanism to address DoS attacks on IoT devices. We propose an anomaly DoS detection technique based on heuristics to tackle SYN, ICMP and UDP flood attacks through the application of machine learning. We evaluate the performance of the proposed technique based on CPU and memory utilization efficiency, power consumption, time taken to detect and defend against a DoS attack. The proposed technique achieves promising results with respect to the key performance indicators used; an efficient CPU utilization, an average of 23 megabytes of memory usage, a power consumption of 35.8mW and an average detection and mitigation time of 0.10 and 0.22 seconds respectively.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

It has been estimated that approximately 50 billion of devices will be connected to the Internet by the end of 2020 [1]. The direct connection of these devices to the Internet makes them vulnerable to several security threats; especially in instances where little or no security measures have been put in place. It has been demonstrated that a number of IoT devices are susceptible to security threats [2] resulting from vulnerabilities such as open telnet ports, outdated firmware, and unencrypted transmission of sensitive data [3]. Some of these threats include the Mirai botnet, also known as *Dyn Attack*, which took the Internet by storm in the late 2016 by overwhelming several high-profile targets with massive Distributed Denial of Service (DDoS) attacks [4]. Many popular websites, including Github, were rendered inaccessible for several hours.

* Corresponding author.
E-mail addresses: jjkponyo@ieee.org (J.J. Kponyo), jay@sperixlabs.org (J.O. Agyemang), klogoselorm@yahoo.com (G.S. Klogo), joshuahoforiboateng@gmail.com (J.O. Boateng).

https://doi.org/10.1016/j.iot.2020.100319
2542-6605/© 2020 Elsevier B.V. All rights reserved.

The prevalence of these threats motivates the development of new techniques for identifying and protecting IoT devices. A number of researchers have developed techniques that help improve the security of IoT devices; such as developing lightweight authentication schemes [5], proposing intrusion detection mechanisms etc. This research work is aimed at preventing DoS attacks on IoT devices. The primary aim of DoS attack is to flood devices in the network with false service requests, thus disrupting the normal operation of the network devices. DoS is an attack targeted at exhausting the computational resources of network devices. DoS attack initiated by multiple agents in a network and from various locations is termed as Distributed Denial of Service (DDoS) attack.

With the evolution of IoT and the large number of devices that form this network, host-based security mechanisms are required to protect these devices. In this study, we propose a host-based DoS anomaly detection and defense mechanism for resource-constrained IoT devices. The research addresses ICMP, UDP and TCP flood attacks and the necessary measures are taken when the attack is identified. Section 1 gives a background to the research. Section 2 reviews related works. The research motivation is described in Section 3. Section 4 describes the approach used in the study. Section 5 discusses the results. Section 6 concludes the research work.

2. Related works

In DDoS, several compromised computers are fired remotely against a target determined by the malicious actor. Thousands or millions of requests are sent to the target until the target suffers intermittencies because of overload or over-traffic. This attack forces end devices to restart their systems, suspend services temporarily, or obstruct communication between users and end systems.

IoT devices create a massive attack platform. For example, an attacker can chain together thousands of connected cameras and generate one of the largest DDoS attacks the world has ever seen. The same vulnerabilities that make IoT devices susceptible to being a part of a botnet make them excellent entry points into data centres and cloud infrastructures. Compromised IoT devices can also be used to disrupt the services of other IoT devices through DDoS attacks.

DDoS attacks can target transmission control protocol (TCP) connections. In this case, the botnet floods the connection request of an end device, without ever coming to an end; hence the communication band of the end system is saturated quickly, making it impossible for any user to access the content. Another type of DDoS attack is user datagram protocol (UDP) and internet control message protocol (ICMP) flood, in which a large number of UDP or ICMP packets are sent, which have the double result of saturating the receiver band and its resources, when trying to process an incoming data. The sender is practically always falsified, with the result that innocent nodes receive responses to packets they never sent. The most used is the SYN flood, in which the target is overawed by a huge number of requests to open TCP connections (SYN packets), which are not completed because the reply packet is sent to the falsified sender, thus leaving the resources of the device exhausted [6].

In addressing the issue of DoS or DDoS, researchers have proposed several techniques. The use of simple threshold-based anomaly detection has been used to discern attack traffic from malicious traffic. Simple threshold-based techniques are prone to incorrectly classifying normal traffic as anomalous traffic and are unable to adapt to the evolving nature of the attacks [7]. Anomaly detection has been used in conventional network intrusion detection systems (NIDS) for detecting unwanted behavior in non-IoT networks. A combination of chaos theory and non-linear time series models have also been used in detecting and mitigating DDoS in the Cloud computing environment [8]. In this approach, the non-linear time series models are used to predict the future network traffic states and further, prediction error is calculated to find out the anomaly in the network traffic.

Software defined networking (SDN) has also been used to monitor network traffic at flow-level granularity [9] to detect DoS attack. Researchers have demonstrated that using flow-based features can be effective in detecting security threats without incurring the high cost of deep-packet inspection [10]. A novel flow table approach has been proposed that prevents OpenFlow switches from overloading; hence mitigating DoS attacks [11]. Apthorpe et al., demonstrated how the finite states of consumer IoT devices can actually be reflected in the repeated temporal structures of send/receive traffic rates to infer consumer usage behavior [12]. Anomaly detection has also been applied in SCADA to determine the unique network traffic patterns of sensors and controllers in infrastructure systems [13,14].

Current detection techniques involve the application of machine learning (ML) in identifying malicious Internet traffic. Network intrusion detection systems (NIDS) now use techniques such as nearest neighbor classifiers, support vector machines, and rule-based schemes like decision tress and random forests in detecting malicious traffic. Machine learning techniques can be used to leverage the unique patterns of IoT network traffic for tasks such as device identification [15]. A machine learning DDoS Detection for consumer IoT devices that uses a machine learning algorithm has been proposed [16]. The focus of their research was on developing a DoS detection technique using low-cost machine learning algorithms for home gateway routers or other network middleboxes. The limitation of their research was on the fact that they used the source internet protocol (IP) address in their ML anomaly detection. Source IP address can be spoofed and as such it should not be used as a feature in developing the ML model. Data mining techniques have also been used in detecting DoS attacks in conventional networks [17]. This approach focused on a network-based DoS detection mechanism. The approach does not provide a host-based DoS detection technique; in instances where the network-based DoS detection technique fails. In SDN, machine learning classification algorithms have also been used in detecting DoS attacks targeted at the controller [18–20].

3. Research motivation

Most DoS detection techniques are network-based with less focus on the end devices. This renders the end devices defenseless when these network-based DoS detection techniques fail. In the context of IoT, the end devices are resource constrained[21]; hence the need for lightweight algorithms. These constraints include low power, memory and processing resources. The Internet Engineering Task Force (IETF) classifies IoT devices into 3; Class 0, 1 and 2. Class 0 consists of IoT devices that do not have the resource to directly implement an IP stack and associated security mechanisms. Devices found in Class 1 are greater than Class 0. They are incapable of communicating with nodes employing a full IP stack. They can however implement an optimized stack specifically designed for constrained nodes. Class 2 devices are characterized by running full implementation of an IP stack on embedded devices [22]; which are the focal point of this research.

Current state of the art security mechanisms cannot be applied on these host devices; hence they are mostly implemented at the border gateway of the network. This emphasizes the need for lightweight security measures that can be implemented on the IoT devices.

This research proposes a lightweight and host-based anomaly DoS detection and defense mechanism (based on NetFilterQueue [28]) for IoT devices. NetfilterQueue is implemented in Linux systems to match packets as accepted, dropped, altered or given a mark. Rules to match packets have to be manually set; which does not make it scalable. Leveraging on this, we implemented the defense mechanism by automatically matching packets based on the output of the detection algorithm. The research contributions of this paper are:

- A lightweight and resource efficient DoS detection algorithm and
- A real-time defense mechanism for embedded IoT devices.

4. Methodology

Our anomaly detection technique is based on observing certain characteristics on the end devices when subjected to a DoS attack. For IoT devices that run the Linux operating system, the kernel provides the following characteristics:

- CPU count; logical CPUs.
- CPU frequency and utilization (%).
- CPU statistics; interrupt requests (IRQ) and system calls (syscalls).
- CPU times; represents the seconds the CPU has spent in a given mode (user space, idle time etc.)
- Memory Utilization (includes virtual/swap memory).
- Load Average; the number of processes in the system run queue averaged over the last 1, 5, and 15 minutes.
- Network connections (system-wide socket connections).
- Network Interface statistics; state, bytes sent / received & maximum transmission unit (MTU).

The following assumptions were made in this research:

- Each IoT device has a particular profile that characterizes its operation as specified by the manufacturer usage description (MUD) specification [23]; which forms the basis of our anomaly detection and defense mechanism.
- The training of the detection engine is done off-device; the model is only exported and used on-device.

4.1. Architecture

A simple local area network (LAN) was used as the base architecture in the research (shown in Fig. 1). The architecture comprised of four (4) IoT devices of which one acted as a malicious node. All the nodes were connected to a gateway router. We used a test bed to evaluate our proposed mechanism.

4.2. Simulation test bed

Four (4) Raspberry Pi [24] devices running Raspbian (a Debian-based operating system for ARM devices) [25] were used as the end devices; one (1) of which acted as a malicious node. Three scenarios were considered for each attack. The scenarios were based on the rate of the flood attacks. ***hping3*** [26] was used in generating ICMP, UDP and TCP flood attacks. The *hping3* tool provides three flood attacks rates; the scenarios were based on these rates. Scenario 1 (S1) has a flood rate of 10 packets/second, Scenario 2 (S2) – 100 packets /second and Scenario 3 is random (as fast as possible; AFAP). ***Scapy*** [27] was used in real-time analysis of network traffic. Preprocessing, in-transit and post-processing of packets were done using ***NetfilterQueue***.

4.3. Simulation parameters

- **DoS Simulation Tool**: hping3.
- **DoS Attacks**: ICMP, UDP and TCP Flood.
- **Nodes**: N1, N2, N3, N4 (Malicious node).
- **Scenarios**: S1 (10pkts/s), S2 (100pkts/s), S3 (Random pkts/s - As fast as possible).

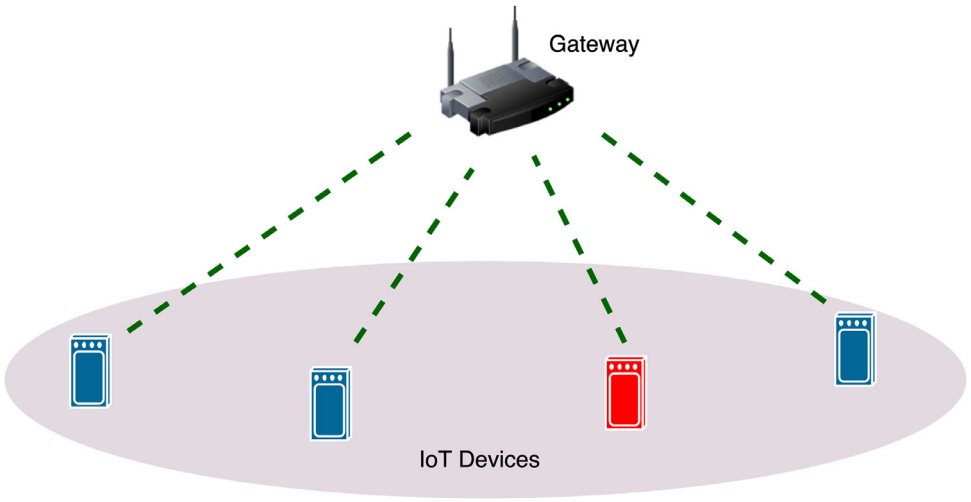


Fig. 1. Network Architecture.

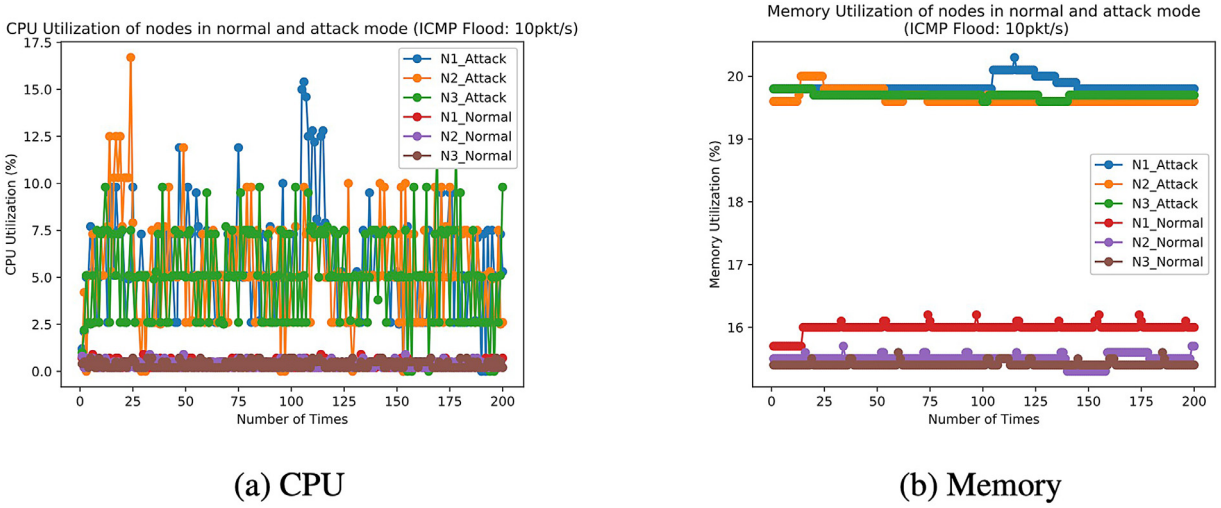


Fig. 2. CPU and Memory utilization of nodes in normal and attack mode (ICMP 10pkt/s).

4.4. Behavioral analysis of DoS traffic

We analyzed the inter-packet arrival times of all the three nodes when subjected to ICMP, UDP and TCP flood attacks. The purpose of this analysis was to determine the behaviour of the flood attacks. To better understand the behaviour of the flood attacks, we correlated the flood arrival rates with the effect it had on the CPU and memory utilization of the end devices. Figs. 2–4 shows the effect of the ICMP flood attack on the CPU and memory for all three scenarios. That of TCP and UDP are shown in Figs. 5–7 and Figs. 8–10 respectively. It can be observed that when the rate of arrival increases, the flood attacks have significant impact on the CPU and memory; hence the basis for using them as heuristics parameters.

We analyzed the interpacket arrival time of the floods to determine the arrival rate boundary. For DoS packet arrival at time $t_1, t_2, t_3, \ldots, t_n$, the average inter-packet arrival time is given by

$$Pkt_{intArr} = \frac{1}{n-1}\sum_{i=1}^{n-1} t_{i+1} - t_i \tag{1}$$

The average inter-packet arrival time of the ICMP, TCP and UDP for all the three scenarios are shown in Figs. 11–13 respectively. S1 in all the three cases had the lowest inter-packet arrival time because the DoS packet rate was 10 packets per second. S2 and S3 had a close inter-packet arrival time due to the fact that S2's DoS packet rate was 100 packets per second and that of S3 was random (as fast as possible).

Empirical analysis of the inter-packet arrival time $\triangle t$ showed that it is bounded within $0 < \triangle t < 0.3$ (shown in Table 1).

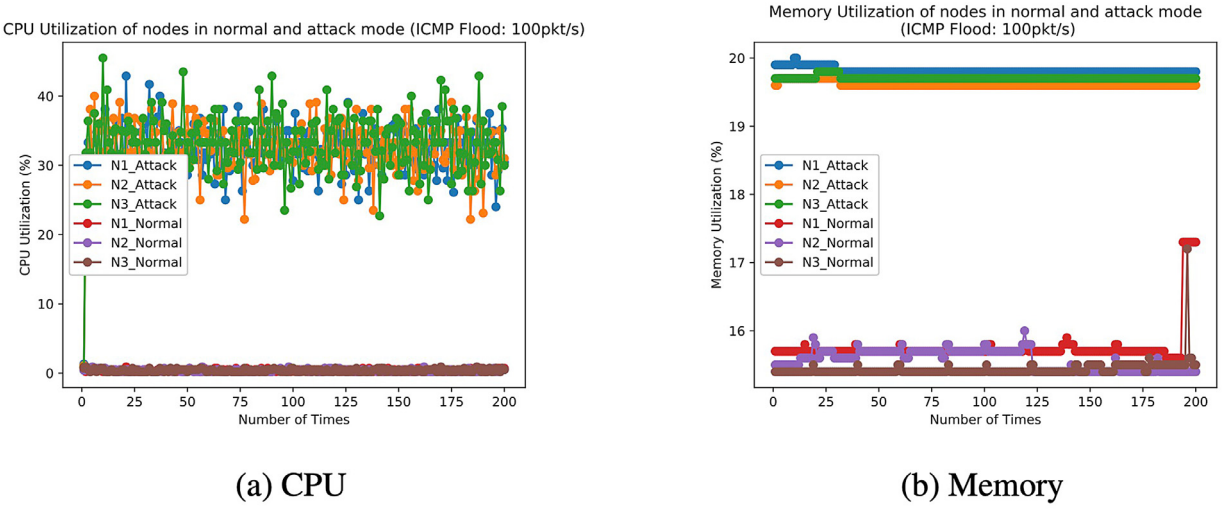


Fig. 3. CPU and Memory utilization of nodes in normal and attack mode (ICMP 100pkt/s).

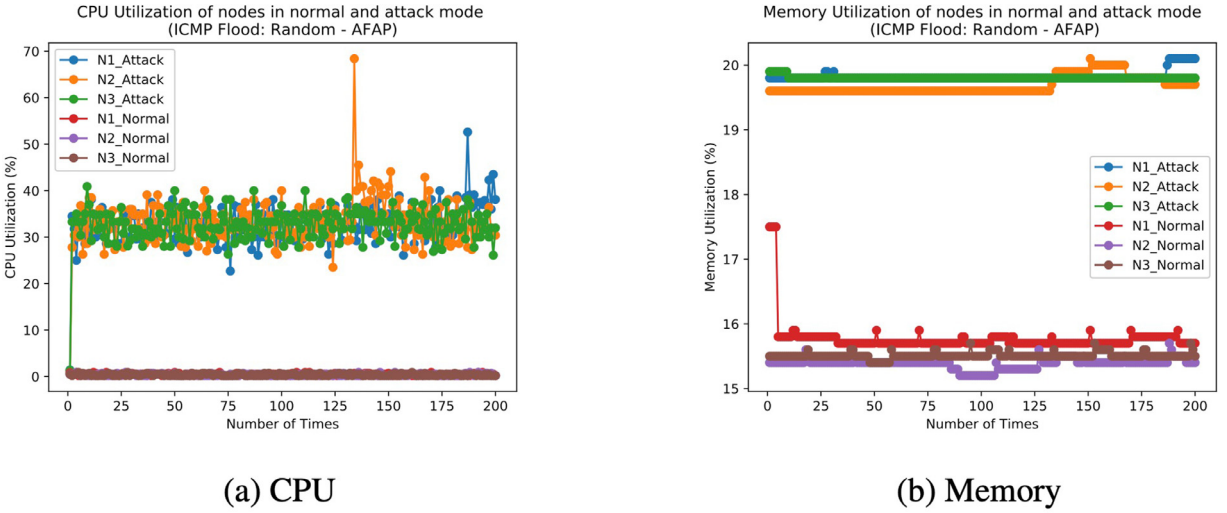


Fig. 4. CPU and Memory utilization of nodes in normal and attack mode (ICMP - As fast as possible).

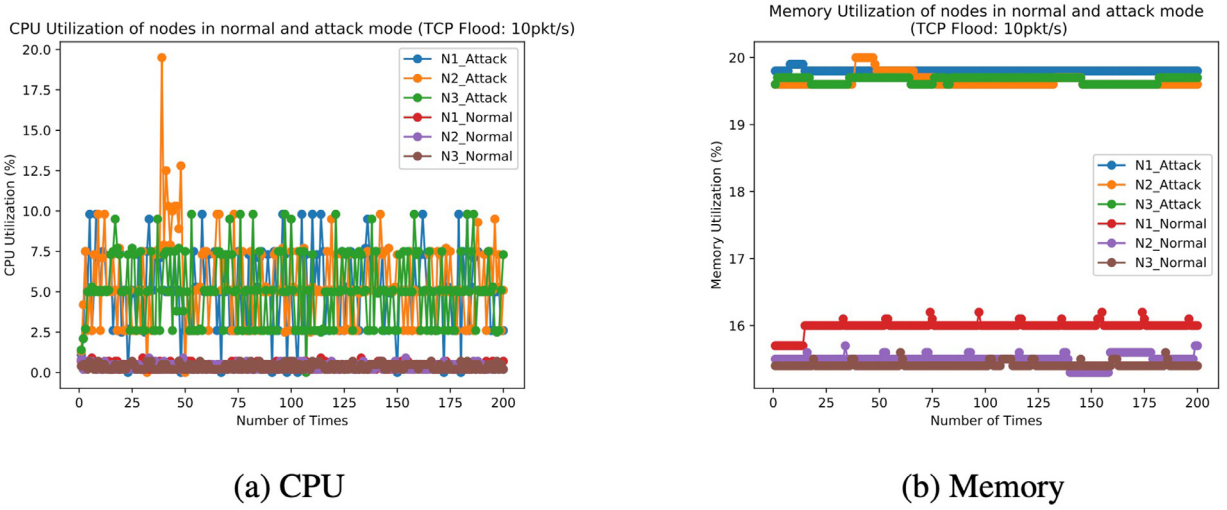


Fig. 5. CPU and Memory utilization of nodes in normal and attack mode (TCP - 10pkt/s).

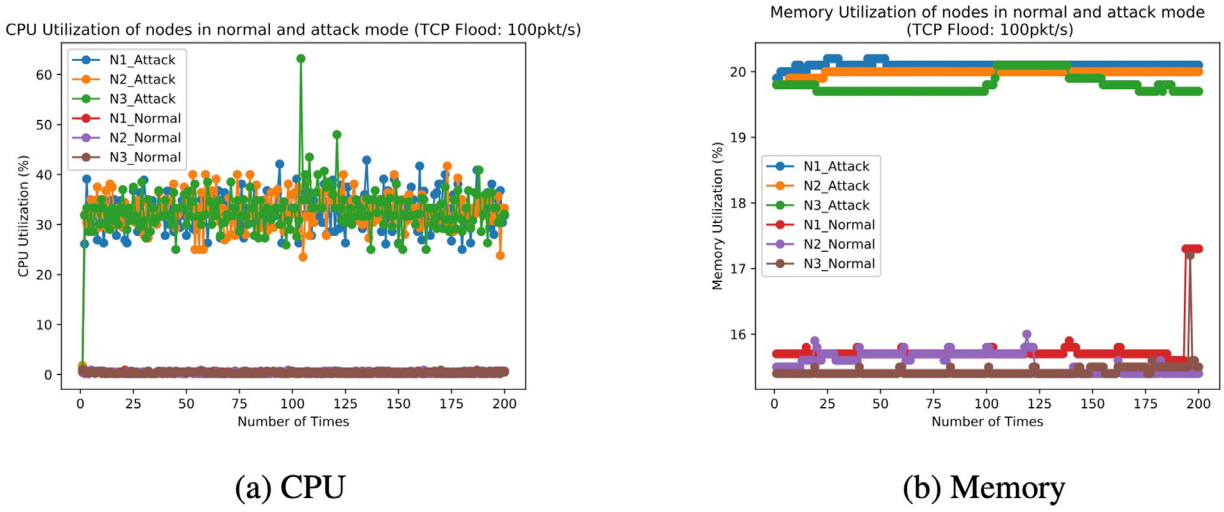


Fig. 6. CPU and Memory utilization of nodes in normal and attack mode (TCP - 100pkt/s).

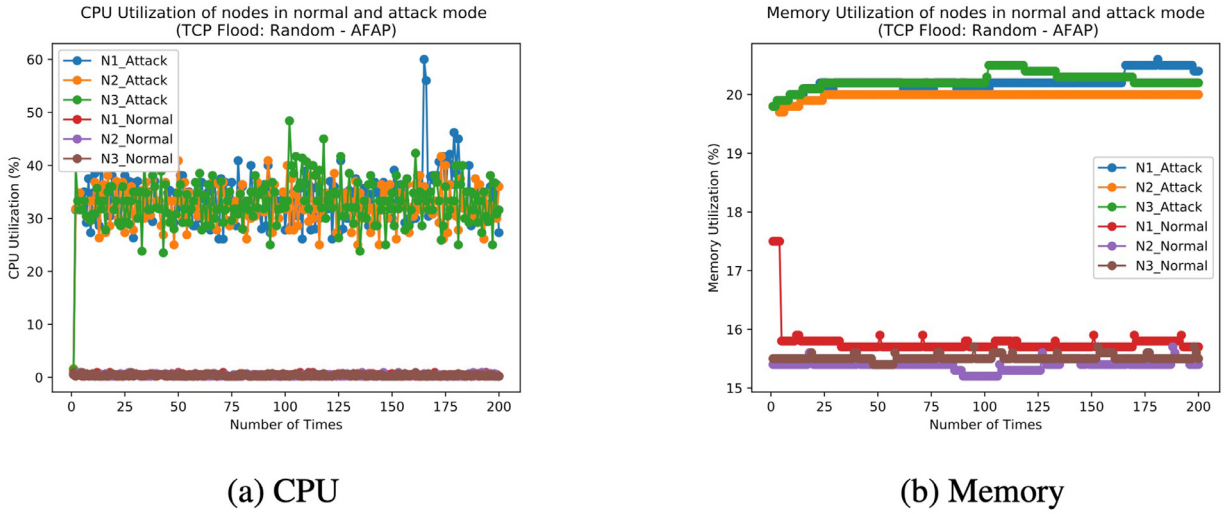


Fig. 7. CPU and Memory utilization of nodes in normal and attack mode (TCP - As fast as possible).

Table 1
Inter-Packet Arrival Time.

Scenarios						
	1		2		3	
	Min	Max	Min	Max	Min	Max
ICMP	0.0972	0.12146	0.04400	0.10576	0.043816	0.11359
TCP	0.0980	0.10396	0.04324	0.12740	0.043760	0.11577
UDP	0.0964	0.20060	0.04380	0.09898	0.043570	0.12289

To detect whether a sequence of packets sent to a node is malicious, the detection agent should be able to observe a number of events and then classify the sequence as a DoS attack or not. We chose the observable window (ω) to be twice the maximum inter-packet arrival time.

$$\omega = 2\triangle t \tag{2}$$

For host-based detection, the agent should be able to observe the influx of packets at regular intervals. The time difference between the start and end of an observation is what we call the *observable window*. Choosing an appropriate observable window is significant because it helps the agent to quantify the influx of packets. Choosing a very small observable window

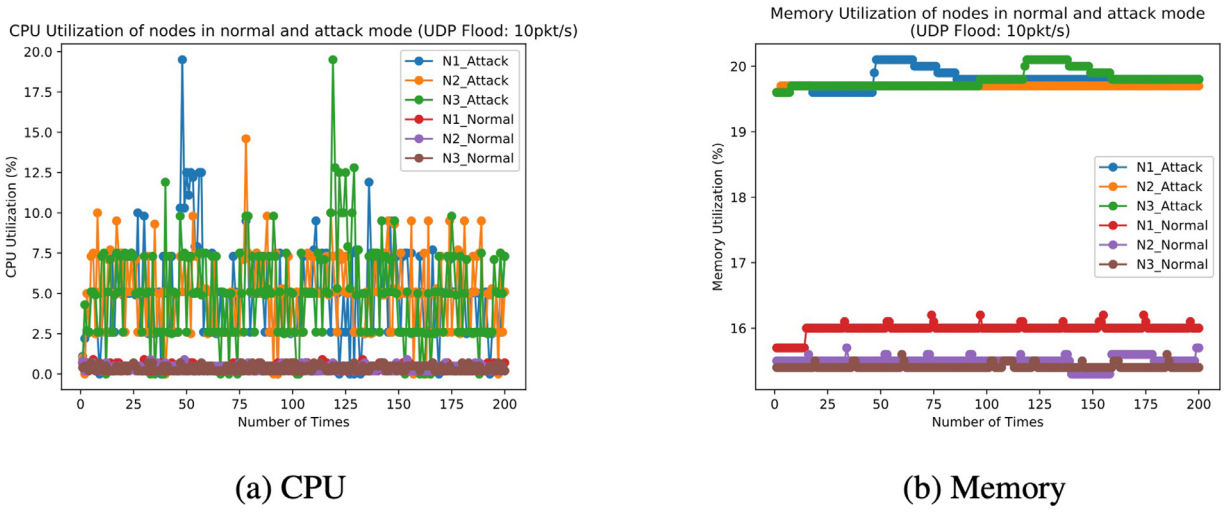


Fig. 8. CPU and Memory utilization of nodes in normal and attack mode (UDP - 10pkt/s).

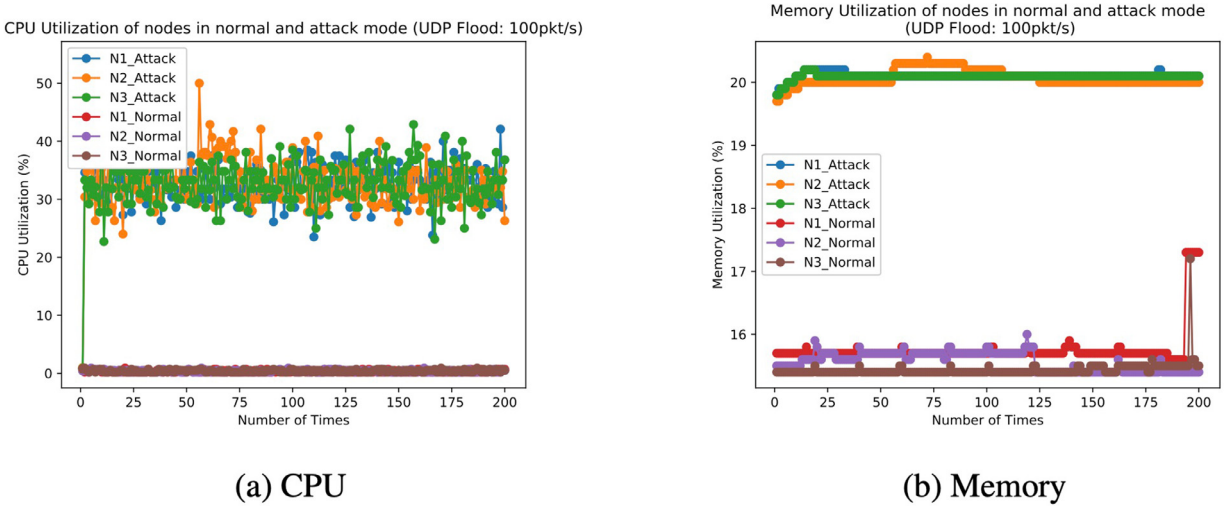


Fig. 9. CPU and Memory utilization of nodes in normal and attack mode (UDP - 100pkt/s).

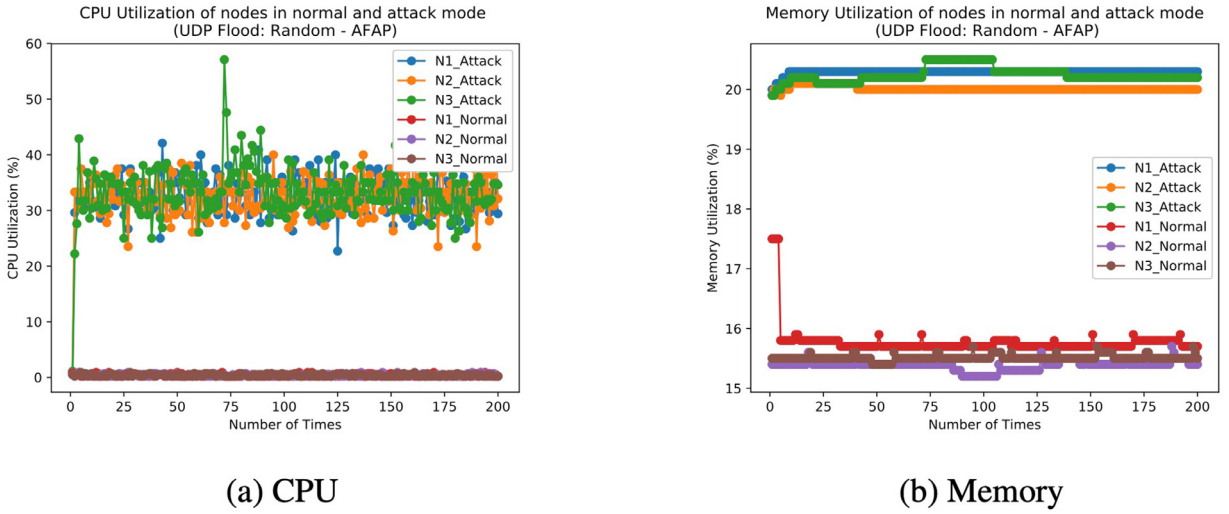


Fig. 10. CPU and Memory utilization of nodes in normal and attack mode (UDP - As fast as possible).

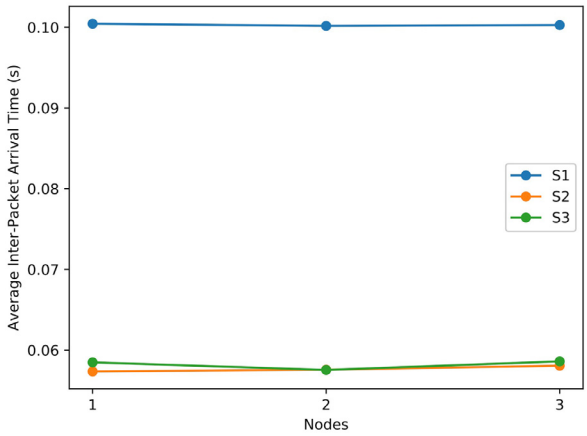


Fig. 11. Average Inter-Packet Arrival Time for ICMP Flood Attack.

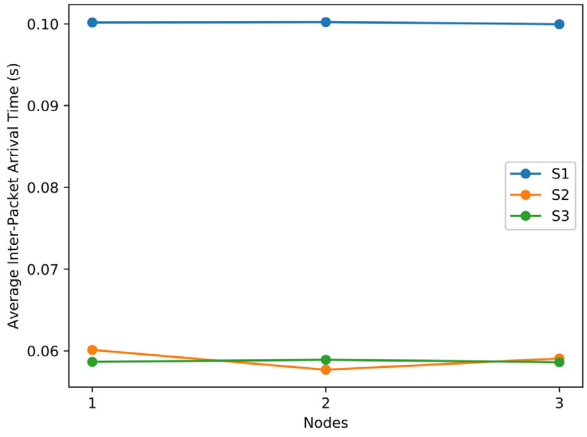


Fig. 12. Average Inter-Packet Arrival Time for TCP Flood Attack.

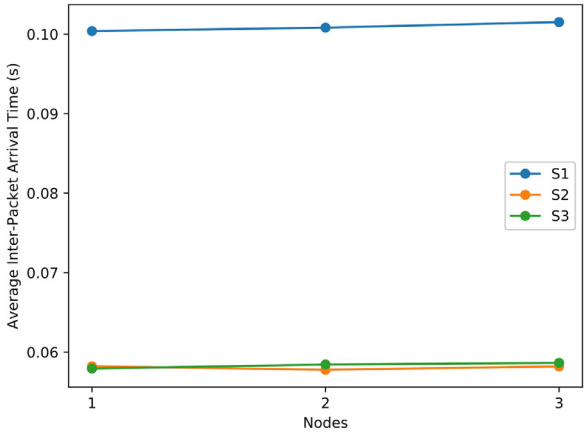


Fig. 13. Average Inter-Packet Arrival Time for UDP Flood Attack.

can cause the agent to misclassify an influx of packets as being a DoS attack. Furthermore, using a large observable window can make the node unable to mitigate against a DoS attack even when it is detected due to its resources being exhausted.

Using $2\triangle t$ as the observable window, we performed an analysis on how much DoS flood packets can be observed within that window and also whether the observed packets would impact the CPU and memory of the end device. Fig. 14 shows the analysis made. It shows that using an observable window of $2\triangle t$ is appropriate because within the window, we can quantify a number of packets arriving correctly without any misclassification. Also, the number of packets that would have

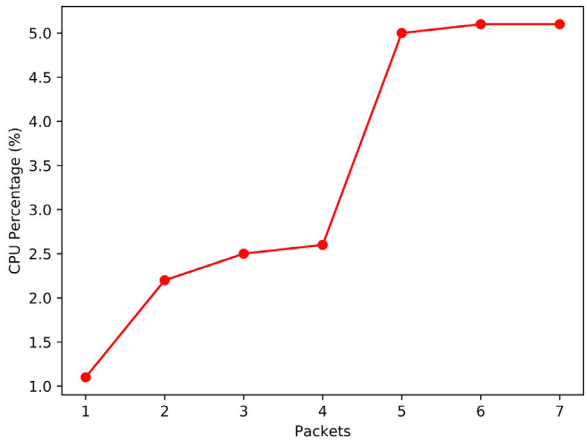


Fig. 14. Empirical analysis to validate our choosing of the observation window $2\triangle t$.

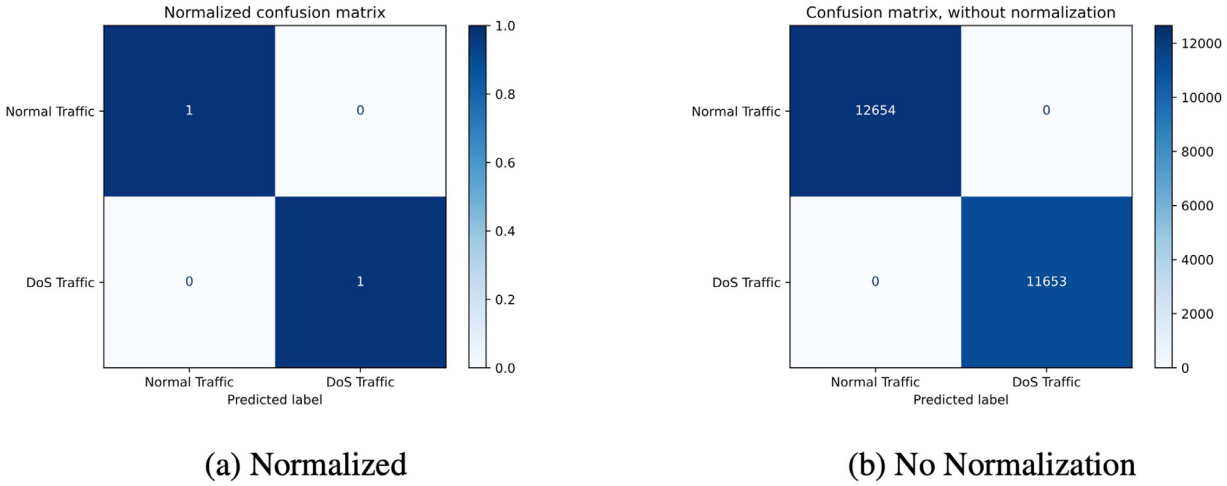


Fig. 15. Confusion Matrix of Training Data.

arrived within that window does not have much significant impact on the node's resource hence making it possible to defend against it when classified as a DoS attack; since one of our core objective is on early detection and mitigation.

4.5. Detection & defense mechanism

The computed observable window ω was used in the detection mechanism. When the devices were subjected to DoS attack, the protocol type and destination port were determined by analyzing the packets in real-time. At that point, the current CPU and memory usage of the device is determined. This is passed to a binary classifier model which was developed using the behavioral profile of the devices when they operated in normal and attack mode. The model was developed using *k*-nearest neighbor classifier. The model classifies packets falling within the observable window as malicious or not based on heuristics (the CPU and memory usage) after observing an influx of packets within the observable window. The total data collected, both normal and malicious was 30384, out of which 14,560 was malicious and 15,824 being normal data. 80% of the data was used for training and the remaining 20% for the evaluation of the model. The confusion matrices of the training and test data are shown in Figs. 15 and 16 respectively.

It can be observed from Figs. 15 and 16 that the model gives a 100% accuracy. This is due to the fact that the model can efficiently distinguish between the normal and DoS traffic based on the chosen heuristic parameters (as shown in Figs. 2–10). The accuracy of the model outperforms those that were used in [8,18–20] which were 99.4%, 97%, 98%, 75.75% respectively.

When a DoS attack is detected (ICMP, TCP or UDP), the protocol type and the destination port is sent to the *NetfilterQueue* engine which enables us to drop packets going to that destination port. It is possible for the source IP address to be spoofed; hence we did not consider dropping packets based on the source IP address. A summary of the entire detection and defense mechanism is shown in Fig. 17.

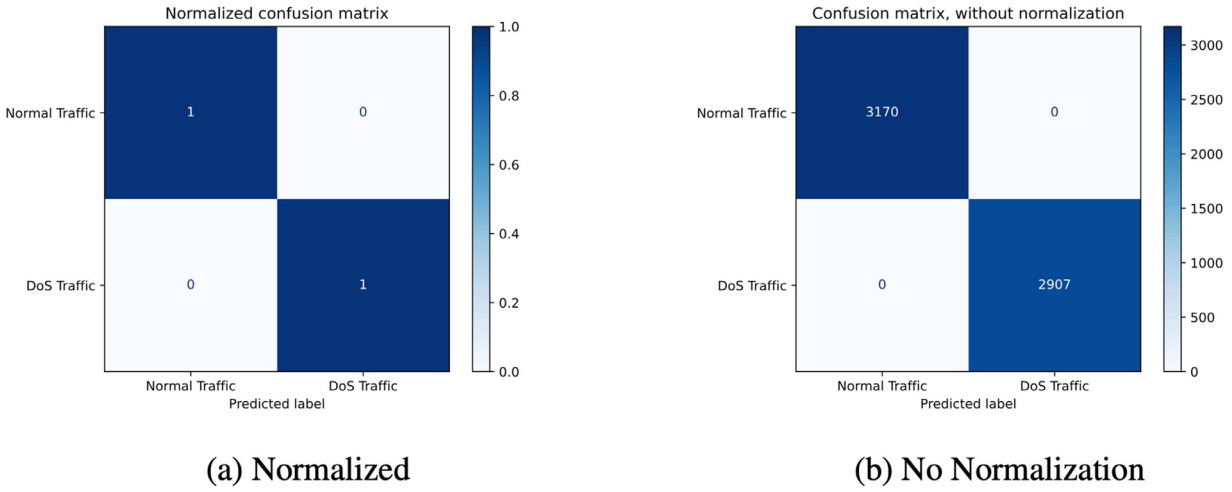


Fig. 16. Confusion Matrix of Test Data.

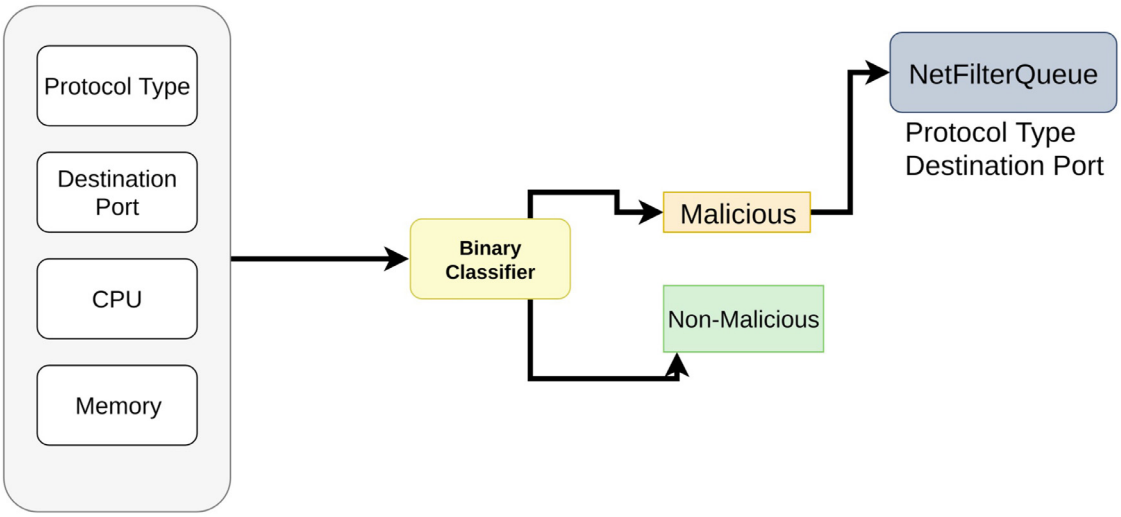


Fig. 17. Decision Model.

5. Results and discussion

We evaluated the proposed DoS detection and defense mechanism based on the CPU and memory utilization efficiency of the proposed technique, the DoS detection and mitigation time.

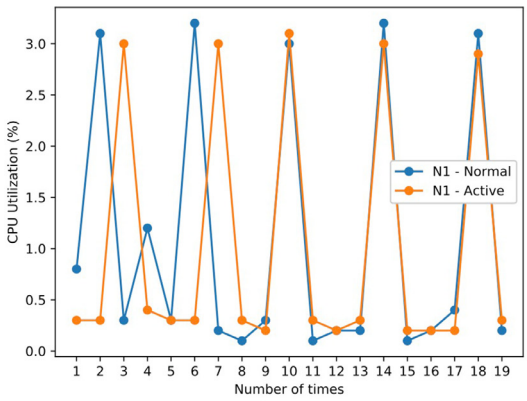
The key performance indicators used in evaluating the proposed DoS detection and defense mechanism were the CPU and memory utilization efficiency of proposed technique, DoS detection and mitigation time.

We measured the CPU and memory utilization of three nodes (N1, N2 and N3) in two modes. The first mode was when the end devices operated normally without the implementation of the proposed detection and defense mechanism. The second mode was when the proposed detection and defense mechanism was implemented.

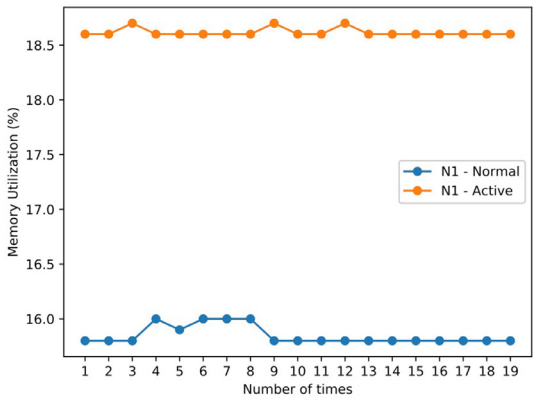
Fig. 18 shows the CPU and memory use of N1. The CPU profile toggles up and down because of the CPU cycles. It can be observed that the proposed detection and defense mechanism does not significantly impact the CPU utilization of the end device. The normal mode had an average memory utilization of 15.8% and the active mode also had an average memory utilization of 18.62%; 2.82% overhead. This shows that the proposed technique is quite memory efficient.

Evaluating the proposed technique on N2 also proves its efficiency as shown in Fig. 19. The average memory utilization in normal mode was 16.05% and in active mode was 18.13%.

Furthermore, the average memory utilization for N3 in normal mode was 16.20% and in active mode was 18.26%. This is shown in Fig. 20.

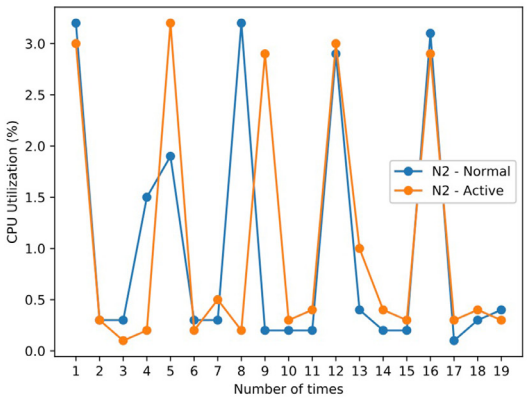


(a) CPU Utilization of N1

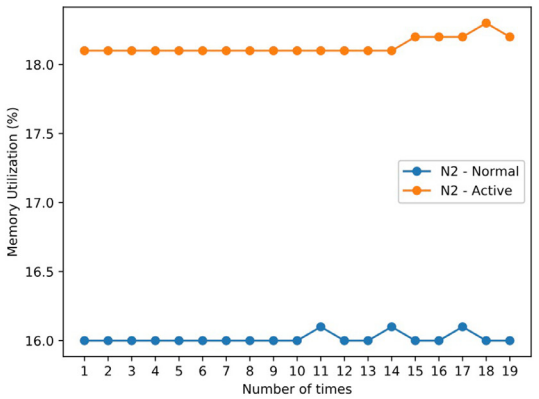


(b) Memory Utilization of N1

Fig. 18. CPU and Memory utilization of N1.

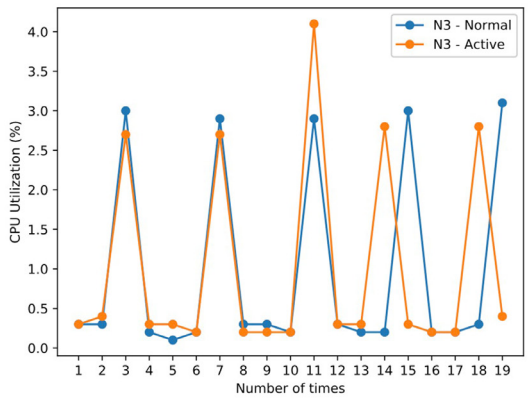


(a) CPU Utilization of N2

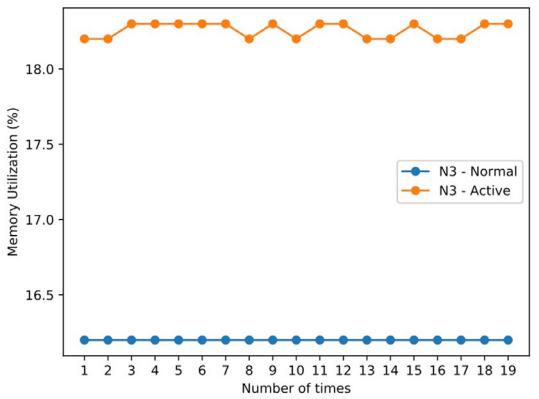


(b) Memory Utilization of N2

Fig. 19. CPU and Memory utilization of N2.



(a) CPU Utilization of N3



(b) Memory Utilization of N3

Fig. 20. CPU and Memory utilization of N3.

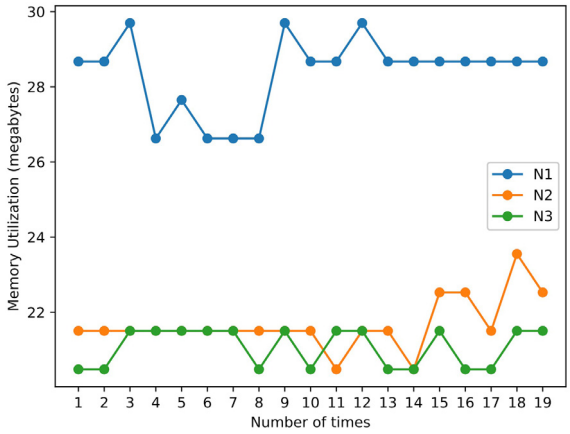


Fig. 21. Memory usage (in megabytes) of proposed algorithm.

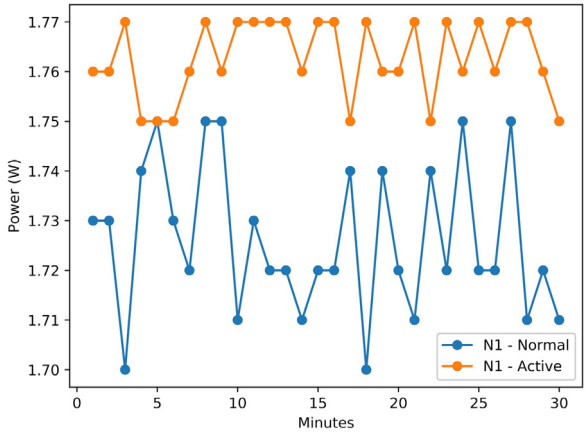


Fig. 22. Power Consumption of N1.

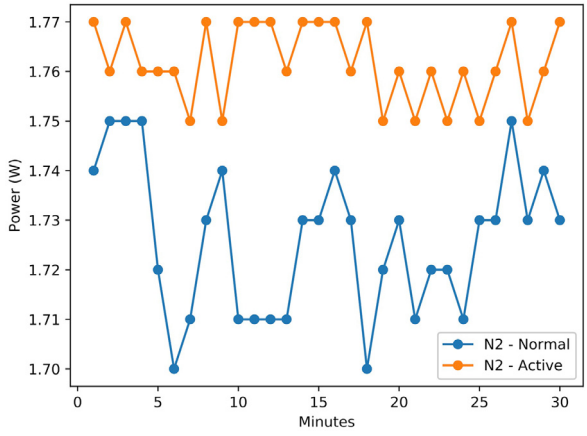


Fig. 23. Power Consumption of N2.

The memory usage (in megabytes) of the proposed algorithm when implemented on the nodes (N1, N2 and N3) is shown in Fig. 21. The average memory utilization of the proposed algorithm for N1, N2 and N3 was 28, 22 and 21 megabytes respectively.

To determine if the proposed algorithm was power efficient, we measured the power consumption (in watts) of the nodes when operating in a normal mode and when in active mode (with the proposed detection algorithm running); as shown in Figs. 22–24. The average power consumption overhead of the algorithm was 0.0358 W (35.8 mW).

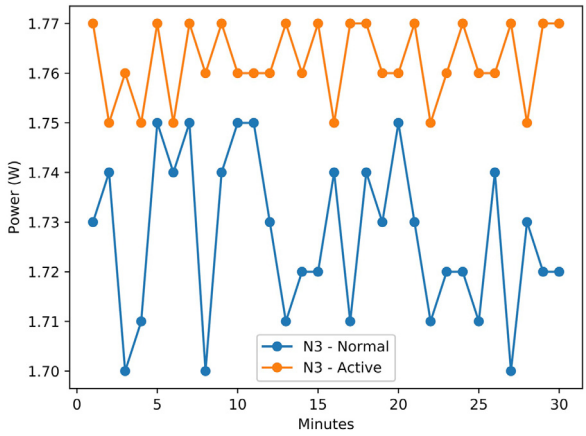


Fig. 24. Power Consumption of N3.

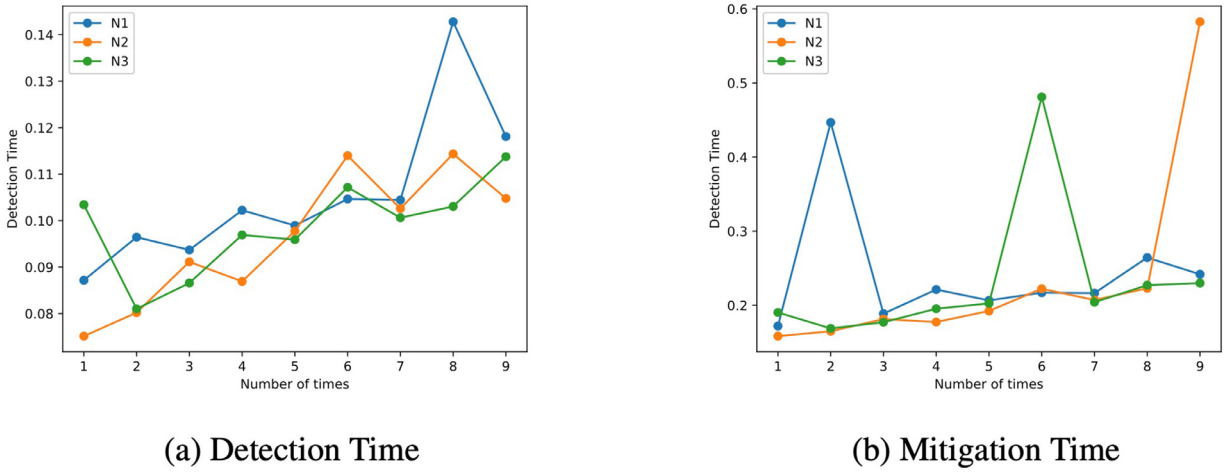


Fig. 25. ICMP detection and mitigation time.

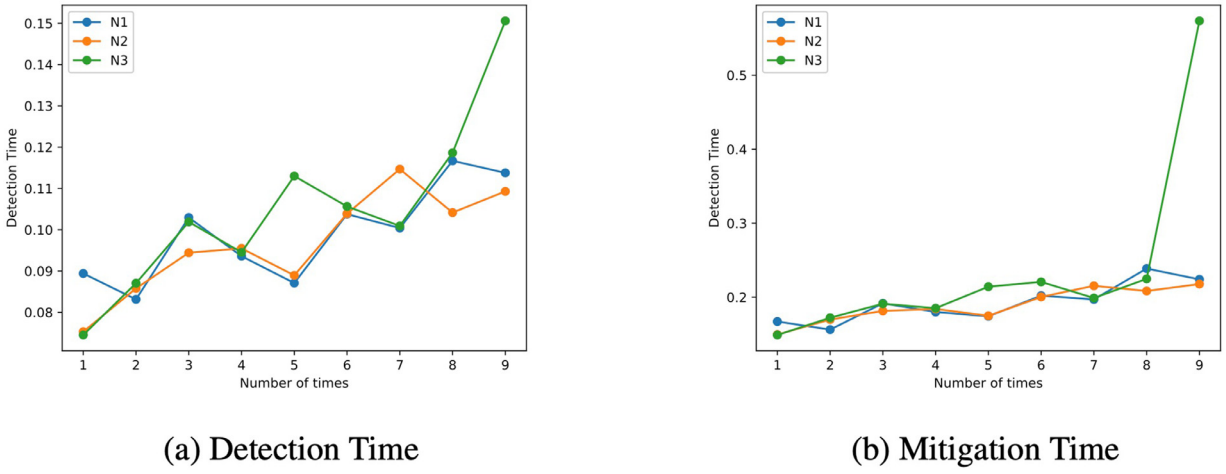


Fig. 26. TCP detection and mitigation time.

We measured the detection and mitigation time for ICMP, TCP and UDP flood attacks for N1, N2 and N3. The average detection time for ICMP flood attack for N1, N2 and N3 was 0.10, 0.09 and 0.11 s respectively. The average time taken to defend ICMP flood attack for N1, N2 and N3 was 0.23, 0.22 and 0.25 s respectively. This is shown in Fig. 25.

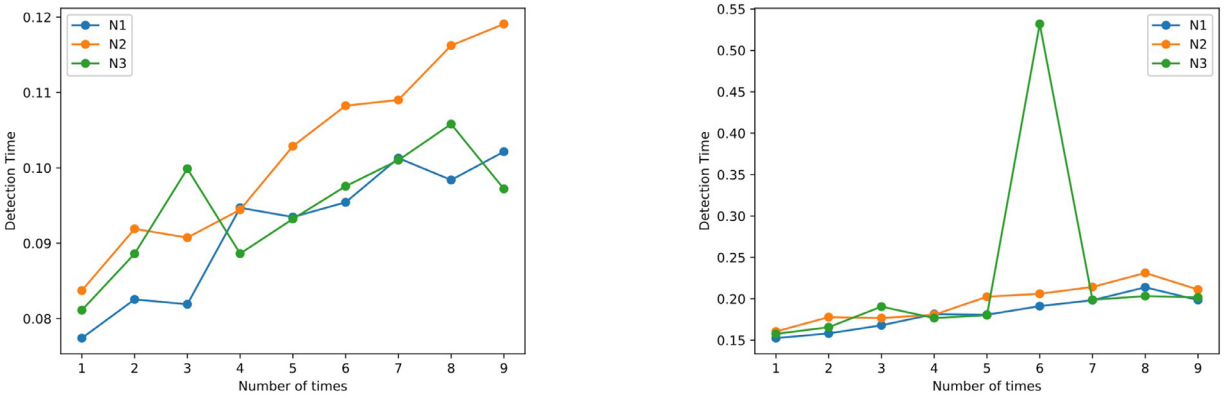


Fig. 27. UDP detection and mitigation time.

Table 2
Comparison of classification algorithm accuracy with related works.

Author	**Classification accuracy**
[8]	99.4%
[18]	97.0%
[19]	98.0%
[20]	75.75%
Proposed model	100%

The average detection time for TCP flood attack for N1, N2 and N3 was 0.098, 0.099 and 0.107 s respectively. The average time taken to defend TCP flood attack for N1, N2 and N3 was 0.219, 0.216 and 0.230 s respectively. This is shown in Fig. 26.

Fig. 27 shows the detection and mitigation time for UDP flood attack. The average detection time for N1, N2 and N3 was 0.093, 0.099 and 0.104 s respectively. The average mitigation time for N1, N2 and N3 was 0.184, 0.192 and 0.218 s respectively.

The proposed technique performs well in detection of DoS attack and also its mitigation. The average time taken to detect the DoS attack is approximately 0.10 s whereas the time taken to defend the attack averages 0.22 s. The proposed technique outperforms conventional techniques such as proposed by Kotey et al. [17]; which had a detection time of 28.9 s.

6. Conclusion and recommendation

In this research, we have proposed a lightweight and host-based DoS anomaly detection and defense mechanism for resource constrained IoT devices. The proposed mechanism proves itself to be resource efficient in terms of CPU and memory use efficiency. Also it has a faster detection and mitigation time as compared to a conventional DoS detection approach [17] which has a detection time of 28.9 seconds and an overhead memory utilization of 2.82%. Furthermore, the binary classification model used in this research achieved a 100% accuracy as compared to those in [8,18–20] whose highest were 99.4%, 97%, 98% and 75.75% respectively (shown in Table 2).

Even though this research produces promising results, it is limited in the sense that it uses off-device training. In instances where the behaviour of the IoT device changes (probably due to a firmware update), the proposed mechanism might not give accurate results. Future works will tackle how federated learning can be used to provide continuous training of the model based on the data gathered from the nodes. We will explore efficient and secure data exfiltration techniques to ensure minimal performance overhead and also protect the integrity of the data.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

Authors would like to acknowledge the support of MTN Ghana in providing funding for this research.

References

[1] D. Evans, The Internet of Things: How the Next Evolution of the Internet is Changing Everything, Cisco Internet Business Solutions Group (IBSG), 2011, p. 3.
[2] C. Kolias, A. Stavrou, J. Voas, I. Bojanova, R. Kuhn, Learning internet-of-things security hands-on, IEEE Secur. Priv. 202–11. 10.1109/MSP.2016.4
[3] Broadband internet technical advisory group (BITAG), internet of things security and privacy recommendations, 2016, http://www.bitag.org/documents/BITAG_Report_-_Internet_of_Things_(IoT)_Security_and_Privacy_Recommendations.pdf.
[4] M. Antonakakis, T. April, Understanding the mirai botnet, in: 26th USENIX Security Symposium, 2017, p. 1.
[5] B. Hammi, A. Fayad, R. Khatoun, S. Zeadally, Y. Begriche, A lightweight ECC-based authentication scheme for internet of things (IoT), IEEE Syst. J.10.1109/JSYST.2020.2970167
[6] U. Ali, A.u. Hassan, M.S.A. Malik, S.K. Hashmi, B. Sslam, B. Ayub, A review of distributed denial of service, Int. J. Comput. Sci.Netw. Secur. 18 (8) (2018).
[7] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, ACM Computing Surveys (CSUR) (15) (2009). 41.3
[8] O.P. Badve, B.B. Gupta, S. Yamaguchi, Z. Gou, DDos detection and filtering technique in cloud environment using GARCH model, in: 2015 IEEE 4th Global Conference on Consumer Electronics (GCCE), Osaka, 2015, pp. 584–586, doi:10.1109/GCCE.2015.7398603.
[9] S.M. &. St-Hilaire, M. Mousavi, Early detection of DDos attacks against SDN controllers, Comput. Netw. Commun. (ICNC) (2015).
[10] A. Sivanathan, D. Sherrat, H.H. Gharakheili, A.V.V. Sivaraman, Low-cost flow-based security solutions for smart-home IoT devices, 2016.
[11] K. Bhushan, B.B. Gupta, Distributed denial of service (DDos) attack mitigation in software defined network (SDN)-based cloud computing environment, J. Ambient Intell. Hum. Comput. 10 (2019) 1985997, doi:10.1007/s12652-018-0800-9.
[12] N. Apthorpe, D. Reisman, N. Feamster, A smart home is no castle: Privacy vulnerabilities of encrypted IoT traffic, 2016.
[13] J. Bigham, D. Gamez, N. Lu, Safeguarding scada systems with anomaly detection, in: V. Gorodetsky, L. Popyack, V. Skormin (Eds.), Computer Network Security, SpringerBerlin Heidelberg, Berlin, Heidelberg, 2003. 171-182
[14] S.N. Shirazi, A. Gouglidis, K.N. Syeda, S. Simpson, A. Mauthe, I.M. Stephanakis, D. Hutchison, Evaluation of anomaly detection techniques for Scada communication resilience, in: 2016 Resilience Week (RWS), 2016, pp. 140–145.
[15] M. Miettinen, S. Marchal, I. Hafeez, N. Asokan, A. Sadeghi, S. Tarkoma, IoT sentinel: automated device-type identification for security enforcement in IoT, CORR (2016) arXiv:1611.04880.
[16] R. Doshi, N. Apthorpe, N. Feamster, Machine learning DDos detection for consumer internet of things devices, arXiv:1804.04159v1
[17] K.S. Djanie, T.E. Tutu, G.J. Dzisi, A proposed dos detection scheme for mitigating dos attack using data mining techniques, Computers 8 (85) (2019), doi:10.3390/computers8040085.
[18] A. Prakash, R. Priyadarshini, An intelligent software defined network controller for preventing distributed denial of service attack, in: Second International Conference on Inventive Communication and Computational Technologies (ICICCT), Coimbatore, 2018, pp. 585–589, doi:10.1109/ICICCT.2018.8473340.
[19] B. He, F. Zou, Y. Wu, Multi-SDN based cooperation scheme for DDos attack defense, in: Third International Conference on Security of Smart Cities, Industrial Control System and Communications (SSIC), Shanghai, 2018, p. 1, doi:10.1109/SSIC.2018.8556830.
[20] T.A. Tang, L. Mhamdi, D. McLernon, S.A.R. Zaidi, M. Ghogho, Deep learning approach for network intrusion detection in software defined networking, in: 2016 International Conference on Wireless Networks and Mobile Communications (WINCOM), Fez, Morocco, 2016, p. 25863, doi:10.1109/WINCOM.2016.7777224.
[21] B.B. Zarpelao, R.S. Miani, C.T. Kawakani, S.C. de Alvarenga, A survey of intrusion detection in internet of things, J. Netw. Comput. Appl. (2017) 2–4.
[22] Request for comments (RFC) 7228, terminology for constrained-node networks, internet engineering task force (IETF), 2014, [Online]. Available https://tools.ietf.org/html/rfc7228.
[23] Request for comments (RFC) 8520, manufacturer usage description specification", internet engineering task force (IETF), 2019, [Online]. Available https://tools.ietf.org/html/rfc8520.
[24] R. Pi, https://www.raspberrypi.org/products/raspberry-pi-3-model-b/.
[25] Raspbian, https://www.raspbian.org/.
[26] hping3, https://github.com/antirez/hping.
[27] Scapy, https://github.com/secdev/scapy.
[28] NetFilterQueue, https://github.com/kti/python-netfilterqueue.